

Attention Operational Math, Worked Out by Hand

THE THREE FORMULAS YOU PUT IN AN RFC — KV SIZING, MLA RATIO, AND THE ROOFLINE VERDICT

What this document does. It collects the operational attention math an inference engineer actually writes down when answering “will this deployment fit, and how many requests per GPU?” Three formulas, each with a worked example and a one-line verdict: (1) per-request KV sizing, (2) the MLA compression ratio, and (3) attention *arithmetic intensity* against the hardware ridge point. We derive the H100 ridge of 295 FLOP/B, show prefill sitting far above it (compute-bound) and decode far below it (memory-bound), and explain why that split dictates which optimization wins. Every figure comes from Appendix A (`m03_t05_attention_op_math_engine.py`).

The one idea. A deployment decision is two numbers: how many *bytes* the KV cache costs (capacity — decides concurrency) and where the workload sits on the *roofline* (compute- vs memory-bound — decides which kernel/feature helps). Prefill is compute-bound (long-context AI $\gg$ ridge), so FA-3’s scheduling wins; decode is memory-bound (AI $\approx O(1) \ll$ ridge), so KV reduction (GQA/MLA/FP8) wins. Pick the lever that matches the binding constraint.

CONCEPTS COVERED, IN ORDER

the roofline ridge point; Formula 1 (KV bytes) with a Llama-3-70B example; Formula 2 (MLA ratio) with a DeepSeek-V3 example; Formula 3 (attention arithmetic intensity) prefill vs decode; the H100 vs H200 thresholds; and the RFC verdict lines.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The ridge point: the threshold everything is measured against	2
2	Formula 1 — Per-request KV size	2
3	Formula 2 — MLA compression ratio	3
4	Formula 3 — Attention arithmetic intensity (prefill)	3
5	Putting it in an RFC	4
6	The argument, at a glance	4
6.1	Equation cheat-sheet	5
	Appendix A — Reference implementation	5

1 The ridge point: the threshold everything is measured against

From Module 1: a workload is memory-bound when its **arithmetic intensity** (AI, FLOPs per byte of HBM traffic) falls below the hardware’s *ridge point* — the AI at which the compute roof and memory roof cross.

Ridge point — compute-bound threshold

$$\text{ridge} = \frac{F_{\text{peak}}}{B_{\text{HBM}}}.$$

H100 and H200 thresholds

$$\text{H100} : \frac{989 \times 10^{12}}{3.35 \times 10^{12}} = 295 \text{ FLOP/B}, \quad \text{H200} : \frac{989 \times 10^{12}}{4.8 \times 10^{12}} = 206 \text{ FLOP/B}.$$

Same compute, more bandwidth $\Rightarrow$ a *lower* ridge: the H200 becomes compute-bound sooner, i.e. it tolerates lower-AI workloads before starving.

Intuition

The ridge is the dividing line. If your AI is above 295 on an H100, you are compute-bound — buy/saturate FLOPs, schedule better (FA-3). If below, you are memory-bound — move fewer bytes (quantize weights, shrink the KV cache). Every attention decision below is “which side of 295 am I on, and how do I act on it.”

2 Formula 1 — Per-request KV size

The single number that decides concurrency on a fixed GPU.

KV bytes per request

$$\text{KV}_{\text{bytes}} = 2 \times L \times H_{kv} \times d \times T \times (\text{dtype bytes}).$$

Llama-3-70B, $T = 8192$, FP16 KV, GQA-8

With $L = 80$, $H_{kv} = 8$, $d = 128$, $T = 8192$, dtype = 2:

$$\text{KV} = 2 \cdot 80 \cdot 8 \cdot 128 \cdot 8192 \cdot 2 = 2,684,354,560 \text{ B} = 2.68 \text{ GBperrequest}. \checkmark$$

On an H100 (80 GB) with AWQ-W4 weights ($\sim 37 \text{ GB}$), $\sim 43 \text{ GB}$ remains for KV $\Rightarrow 43/2.68 \approx \mathbf{16}$ concurrent requests at 8K. Switching to FP8 KV (dtype = 1) halves the cache to 1.34 GB $\Rightarrow \sim 32$ concurrent.

Verdict line

“Llama-3-70B GQA-8 @ 8K, FP16 KV = 2.68 GB/req; ~ 16 reqs/H100; FP8 KV doubles to ~ 32 .” This is the line that decides TP=1 vs TP=2 vs “does not fit.”

3 Formula 2 — MLA compression ratio

MLA (Topic 4) replaces per-head K, V with one shared latent. The ratio versus MHA is what produces the cited “ $\sim 98\%$ reduction.”

MLA vs MHA KV ratio

$$\text{ratio} = \frac{c_{\text{dim}} + \text{rope}}{2 \times H_q \times d}.$$

DeepSeek-V3: $c_{\text{dim}}=512$, $\text{rope}=64$, $H_q=128$, $d=128$

$$\text{ratio} = \frac{512 + 64}{2 \cdot 128 \cdot 128} = \frac{576}{32,768} = 0.0176.$$

MLA stores 1.76% of MHA $\Rightarrow 1 - 0.0176 = \mathbf{98.2\%}$ reduction versus full MHA. ✓ (The paper’s “93%” compares against a GQA-like baseline; versus *full* MHA the reduction is 98.2%.)

Verdict line

“DeepSeek-V3 MLA ratio 0.0176 $\Rightarrow$ 98.2% KV reduction vs MHA” — a bigger memory win than any quantization, and the reason 128K context fits on a single node.

4 Formula 3 — Attention arithmetic intensity (prefill)

Where attention sits on the roofline. Numerator: the FLOPs of $QK^\top + \text{softmax} + PV$. Denominator: the bytes read for Q, K, V (the $N \times N$ scores stay in SRAM via FlashAttention, so they are not counted).

Attention arithmetic intensity, prefill (approx.)

$$\text{AI}_{\text{prefill}} \approx \frac{T_q T_k H d \cdot 4}{(T_q H d + T_k H_{kv} d \cdot 2) \cdot (\text{dtype})}.$$

Llama-3-8B prefill, $H_q=32$, $H_{kv}=8$, $d=128$

At $T = 4096$ (self-attention, $T_q = T_k = 4096$):

$$\text{FLOPs} \approx 4096^2 \cdot 32 \cdot 128 \cdot 4 = 2.75 \times 10^{11}, \quad \text{bytes} \approx 50.3 \text{ MB}, \quad \text{AI} \approx \mathbf{5461} \text{ FLOP/B}.$$

$5461 \gg 295 \Rightarrow$ **compute-bound**: FA-3’s warp-specialized, async-GEMM scheduling is what wins here. At $T = 128$ instead:

$$\text{AI} \approx \mathbf{171} \text{ FLOP/B} < 295 \Rightarrow \mathbf{\text{memory-bound}},$$

and FA-3’s compute advantage disappears. This is exactly why FA-3 benchmarks show $\sim 2\times$ at long context and near-zero gain at short context.

The opposite regime: decode AI is $O(1)$

In decode the query is a single row ($T_q = 1$) attending over T_k cached tokens. The FLOPs ($\sim 4T_k H d$) and the bytes (read the whole KV cache, $\sim T_k H_{kv} d \cdot 2 \cdot \text{dtype}$) *both* scale with

T_k , so their ratio is constant:

$$\text{AI}_{\text{decode}} \approx \frac{4T_k H d}{2T_k H_{kv} d \cdot 2} = \frac{H}{H_{kv}} = \frac{32}{8} = 4.0 \text{ FLOP/B} \quad (\text{independent of context length}).$$

$4 \ll 295 \Rightarrow$ deeply **memory-bound**, at any context length.

The split that decides everything

Prefill (long context): $\text{AI} \gg \text{ridge} \Rightarrow \text{compute-bound} \Rightarrow$ optimize compute (FA-3, better GEMM scheduling). **Decode:** $\text{AI} \approx O(1) \ll \text{ridge} \Rightarrow \text{memory-bound} \Rightarrow$ optimize bytes (GQA, MLA, FP8 KV, quantized weights). Choosing the wrong lever — e.g. buying FLOPs for a decode-bound service — spends effort on the slack constraint, not the binding one.

5 Putting it in an RFC

The three formulas combine into the deploy paragraph:

Question	Formula	Verdict (this config)
Will it fit?	$\text{KV} = 2LH_{kv}dT(\text{dtype})$	2.68 GB/req, $\sim 16/\text{H100}$
Worth MLA?	$(c_{\text{dim}} + \text{rope}) / (2H_q d)$	$0.0176 \Rightarrow 98.2\%$ less KV
Which kernel?	$\text{AI vs ridge} = F_{\text{peak}} / B_{\text{HBM}}$	prefill $5461 \gg 295$ (compute); decode $4 \ll 295$ (memory)

Watch out

Arithmetic intensity for attention ignores the $N \times N$ score matrix in the byte count *only because* FlashAttention keeps it in SRAM. On a naive (non-fused) kernel the scores are written to and read from HBM, the denominator explodes, and AI collapses — attention becomes memory-bound even in prefill. The roofline verdict above assumes a fused FA-style kernel; state that assumption in the RFC.

6 The argument, at a glance

#	Quantity	Result
0	Ridge	$\text{H100} = 295, \text{H200} = 206 \text{ FLOP/B}$
1	KV size	Llama-3-70B GQA-8 @ 8K = 2.68 GB, $\sim 16/\text{H100}$ ✓
2	MLA ratio	DeepSeek-V3 = $0.0176 \Rightarrow 98.2\%$ less ✓
3	Prefill AI	$T=4096: 5461 \gg 295 \Rightarrow \text{compute-bound}$
3	Decode AI	$H/H_{kv} = 4 \ll 295 \Rightarrow \text{memory-bound}$

6.1 Equation cheat-sheet

Ridge	$F_{\text{peak}}/B_{\text{HBM}}$
KV bytes	$2 L H_{kv} d T (\text{dtype})$
MLA ratio	$(c_{\text{dim}} + \text{rope})/(2 H_q d)$
Attn AI (prefill)	$T_q T_k H d \cdot 4 / [(T_q H d + T_k H_{kv} d \cdot 2)(\text{dtype})]$
Attn AI (decode)	$\approx H/H_{kv} \text{ (const in } T)$
Bound test	$\text{AI} < \text{ridge} \Rightarrow \text{memory-bound}$

Appendix A — Reference implementation

Every number above is produced by `m03_t05_attention_op_math_engine.py`. It evaluates all three formulas, both roofline regimes, and prints the RFC verdict lines.

```

1  """
2  Reference implementation for: Module 03 - Topic 5
3  "Attention Operational Math, Worked Out by Hand."
4
5  The three formulas you put in an RFC:
6  (1) per-request KV sizing,
7  (2) MLA compression ratio,
8  (3) attention arithmetic intensity vs the H100 ridge (295 FLOP/B).
9
10 Convention: 1 GB = 1e9 B, 1 MB = 1e6 B (decimal).
11 Every number printed here is transcribed (rounded) into the worked-by-hand PDF.
12 """
13
14 # -----
15 # 0. Hardware ridge points (compute-bound threshold = F_peak / B_hbm).
16 # -----
17 F_peak_h100 = 989e12      # H100 FP16 tensor-core peak, FLOP/s
18 B_h100      = 3.35e12     # H100 HBM bandwidth, B/s
19 F_peak_h200 = 989e12
20 B_h200      = 4.8e12
21
22 ridge_h100 = F_peak_h100 / B_h100
23 ridge_h200 = F_peak_h200 / B_h200
24 print("== 0. Roofline ridge points (FLOP/byte) ==")
25 print(f"H100: {F_peak_h100:.2e} / {B_h100:.2e} = {ridge_h100:.0f} FLOP/B")
26 print(f"H200: {F_peak_h200:.2e} / {B_h200:.2e} = {ridge_h200:.0f} FLOP/B")
27
28 # -----
29 # 1. Formula 1 -- per-request KV size.
30 #   KV = 2 * L * H_kv * d * T * dtype_bytes
31 #   Worked example: Llama-3-70B, 8K context, FP16 KV, GQA-8.
32 # -----
33 def kv_bytes(L, H_kv, d, T, dtype_b):
34     return 2 * L * H_kv * d * T * dtype_b
35
36 L, H_kv, d, T, dtype_b = 80, 8, 128, 8192, 2
37 kv = kv_bytes(L, H_kv, d, T, dtype_b)
38 print("\n== 1. Formula 1: per-request KV size ==")
39 print(f"KV = 2 * {L} * {H_kv} * {d} * {T} * {dtype_b} = {kv:,d} B = {kv/1e9:.2f} GB")
40
41 # Concurrency on one H100 (80 GB) with AWQ-W4 weights ~37 GB.
42 hbm, weights = 80e9, 37e9
43 free = hbm - weights
44 print(f"H100 80GB - 37GB weights = {free/1e9:.0f} GB free -> ~{free/kv:.0f} concurrent reqs (FP16 KV)")

```

```

45 kv_fp8 = kv_bytes(L, H_kv, d, T, 1)
46 print(f"FP8 KV (dtype=1): {kv_fp8/1e9:.2f} GB/req -> ~{free/kv_fp8:.0f} concurrent reqs (doubles)")
47
48 # -----
49 # 2. Formula 2 -- MLA compression ratio.
50 #   ratio = (c_dim + rope_dim) / (2 * H_q * d)
51 #   Worked example: DeepSeek-V3 (128 heads, d=128, c_dim=512, rope=64).
52 # -----
53 c_dim, rope_dim, H_q, d_mla = 512, 64, 128, 128
54 ratio = (c_dim + rope_dim) / (2 * H_q * d_mla)
55 print("\n== 2. Formula 2: MLA compression ratio ==")
56 print(f"ratio = ({c_dim} + {rope_dim}) / (2 * {H_q} * {d_mla}) = {c_dim+rope_dim} / {2*H_q*d_mla} = {ratio:.4f}")
57 print(f"> MLA stores {ratio*100:.2f}% of MHA; {(1-ratio)*100:.1f}% reduction vs full MHA")
58
59 # -----
60 # 3. Formula 3 -- attention arithmetic intensity (prefill).
61 #   AI ~ (T_q * T_k * H * d * 4) / ((T_q*H*d + T_k*H_kv*d*2) * dtype_bytes)
62 #   Worked example: Llama-3-8B prefill, H_q=32, H_kv=8, d=128.
63 # -----
64 def attn_ai(T_q, T_k, H, H_kv, d, dtype_b=2):
65     flops = T_q * T_k * H * d * 4
66     bytes_io = (T_q * H * d + T_k * H_kv * d * 2) * dtype_b
67     return flops, bytes_io, flops / bytes_io
68
69 print("\n== 3. Formula 3: attention arithmetic intensity (prefill) ==")
70 H8, Hkv8, d8 = 32, 8, 128
71 for Tq in (4096, 128):
72     flops, byts, ai = attn_ai(Tq, Tq, H8, Hkv8, d8)
73     bound = "COMPUTE" if ai > ridge_h100 else "MEMORY"
74     print(f"  Llama-3-8B prefill T={Tq:>5}: FLOPs={flops:.2e} bytes={byts/1e6:.1f} MB "
75           f"AI={ai:7.0f} FLOP/B -> {bound}-bound (vs ridge {ridge_h100:.0f})")
76
77 # -----
78 # 4. Decode is the opposite: AI ~ 0(1), deeply memory-bound.
79 #   One query [1 x d] over T_k cached tokens; FLOPs ~ 4*T_k*H*d,
80 #   bytes ~ T_k*H_kv*d*2 (read the whole KV cache).
81 # -----
82 print("\n== 4. Decode arithmetic intensity (memory-bound) ==")
83 def decode_ai(T_k, H, H_kv, d, dtype_b=2):
84     flops = 4 * T_k * H * d # QK^T + PV over the cache, 1 query
85     byts = T_k * H_kv * d * 2 * dtype_b # read K and V for all cached tokens
86     return flops, byts, flops / byts
87 for Tk in (6200, 32768):
88     flops, byts, ai = decode_ai(Tk, H8, Hkv8, d8)
89     bound = "COMPUTE" if ai > ridge_h100 else "MEMORY"
90     print(f"  decode, T_k={Tk:>6}: AI={ai:5.2f} FLOP/B -> {bound}-bound "
91           f"f("<< ridge {ridge_h100:.0f}); independent of context length")
92
93 # -----
94 # 5. The verdict lines used in the document.
95 # -----
96 print("\n== 5. RFC verdict lines ==")
97 print(f"- KV sizing: Llama-3-70B GQA-8 @ 8K FP16 = {kv/1e9:.1f} GB/req; ~{free/kv:.0f} reqs/H100.")
98 print(f"- MLA: DeepSeek-V3 ratio {ratio:.4f} -> {(1-ratio)*100:.1f}% KV reduction.")
99 print(f"- AI: prefill long-context AI >> {ridge_h100:.0f} (compute-bound); decode AI ~ 0(1) (memory-bound).")

```